COURSE: WEB SECURITY & PENETRATION TESTING

LAB ASSIGNMENT REPORT: DAY 36 ERROR HANDLING

Student Name: Jiwosh Shrestha	**Topic:** Exception Handling & Secure Error Design
Assignment: Day 36 - Error Handling	**Focus:** OWASP A05 - Security Misconfiguration / Info Disclosure

PART 1: EXCEPTION HANDLING & SECURE ERROR DESIGN

Question 1: What is an exceptional condition? Give four examples.

An exceptional condition is an abnormal runtime situation or unexpected event that disrupts the normal sequential execution flow of a program and requires explicit handling mechanisms.

Four Examples:
1. Attempting to open or read a file that does not exist (FileNotFoundException).
2. Database connection timeout or network socket refusal.
3. Null reference / null-pointer dereference.
4. Arithmetic overflow or division by zero.

Question 2: Explain the difference between Fail Open and Fail Closed / Fail Secure. Give one security example.

• Fail Open: When a system check or component encounters an error, the system defaults to allowing the action or granting access (permissive stance).
• Fail Closed (Fail Secure): When a system check or component encounters an error, the system defaults to denying access or halting the action (restrictive stance).

Security Example: An authentication service or IAM directory that experiences a lookup timeout must Fail Closed (deny login) rather than Fail Open (grant unauthenticated access).

Question 3: Explain why verbose error messages and stack traces may create security risk. List four examples of information that should not normally be exposed to users.

Verbose error messages and stack traces expose internal architectural details that allow attackers to map the application stack, identify underlying software frameworks, locate source code file structures, and execute highly targeted exploits.

Four Examples of Information That Must Not Be Exposed:
1. Absolute server file-system paths (e.g., /home/carlos/applications/product-catalog/)
2. Database table names, column structures, or raw SQL queries
3. Framework, application server, or library version numbers (e.g., Spring Boot 2.1.1, Struts 2.3.31)
4. Internal hostnames, IP addresses, environment variables, or configuration secrets

Question 4: Explain why rollback is important when a multi-step transaction fails.

A multi-step transaction consists of several interrelated database updates or state changes that must execute atomically (all or nothing). If an exception occurs midway through execution and preceding steps remain committed, the application is left in an inconsistent or corrupted state.

Rollback ensures that upon failure, all partial changes are completely reversed, restoring the database to its last known consistent state and maintaining data integrity.

Question 5: Explain how poor resource cleanup during repeated exceptions may affect application availability.

When an exception occurs and open file handles, database connections, network sockets, or memory locks are not explicitly released in a try/finally block, each failed request permanently leaks a system resource.

Repeated exceptions gradually exhaust the available connection or thread pool. Once exhausted, subsequent legitimate user requests cannot acquire resources, leading to system crash or Denial of Service (Resource Exhaustion DoS).

Question 6: Write at least six recommendations for secure exception/error handling.

1. Catch Specific Exceptions: Prefer specific exception types over broad catch-all blocks.
2. Generic Client Responses: Never return stack traces, SQL fragments, file paths, or version banners to clients.
3. Server-Side Logging: Log full diagnostic detail and stack traces exclusively on secure server-side logs.
4. Correlation IDs: Return generic user-facing error pages containing unique correlation reference IDs for support tracking.
5. Guaranteed Resource Cleanup: Always release resources (connections, files, locks) using try-finally or try-with-resources patterns.
6. Fail Closed Design: Design security-critical authorization and authentication checks to Fail Closed.
7. Atomic Rollback: Wrap multi-step operations in database transactions with automatic rollback on error.

PART 2: PRACTICAL EVIDENCE - INFORMATION DISCLOSURE IN ERROR MESSAGES

Question 7: Explain what information was disclosed in the Day 36 PortSwigger lab. Attach evidence and explain how it should be fixed.

In the PortSwigger 'Information disclosure in error messages' lab, submitting an invalid or non-integer product ID parameter triggered an unhandled exception. The application returned an HTTP 500 Verbose Error Page exposing sensitive infrastructure details.

BEFORE - Vulnerable Response (Information Disclosure Evidence):

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/html; charset=UTF-8

java.sql.SQLException: Invalid product ID
  at com.portswigger.lab.ProductDao.FindById(ProductDao.java:42)
  at com.portswigger.lab.ProductController.getProduct(ProductController.java:178)

Root path: /home/carlos/applications/product-catalog/
Framework: Spring Boot 2.1.1
Java 11.0

Information Disclosed:
• Absolute filesystem path
• Internal Java package and class names
• Line numbers in source code files
• Spring Boot & Java runtime version numbers
```

AFTER - Secure Response (Remediation Applied):

```
HTTP/1.1 500 Internal Server Error
Content-Type: text/html; charset=UTF-8

An unexpected error occurred. Please try again later.
(Reference Code: ERR-7F3A9C)
```

How the Fix Works:

1. Centralized Exception Handler: Implements a global exception handler to intercept all unhandled runtime exceptions.
2. Protected Internal Logging: Writes full stack traces and technical diagnostics exclusively to internal, protected log files.
3. Generic Response: Supplies the client with a generic, sanitized HTTP 500 response alongside a unique tracking reference ID.